

PROGRAMAÇÃO EM PYTHON



PROGRAMAÇÃO EM PYTHON



Por meio de uma parceria com o SENAI-SP, a Prefeitura de Santana de Parnaíba está disponibilizando um curso focado em Python, com o objetivo de preparar os alunos para o desenvolvimento de programas e aplicações.



PROGRAMAÇÃO EM PYTHON



Plano de Aula

- Conteúdo:**
- APIs (Application Programming Interface)
 - O que são APIs e como consumi-las.
 - API REST

Início:

As informações deste conteúdo visam compreender o conteúdo do curso.



PROGRAMAÇÃO EM PYTHON



Objetivo

Capacitar profissionais para desenvolver aplicações em linguagem Python, por meio de técnicas de programação, seguindo boas práticas, procedimentos e normas.

PROGRAMAÇÃO EM PYTHON



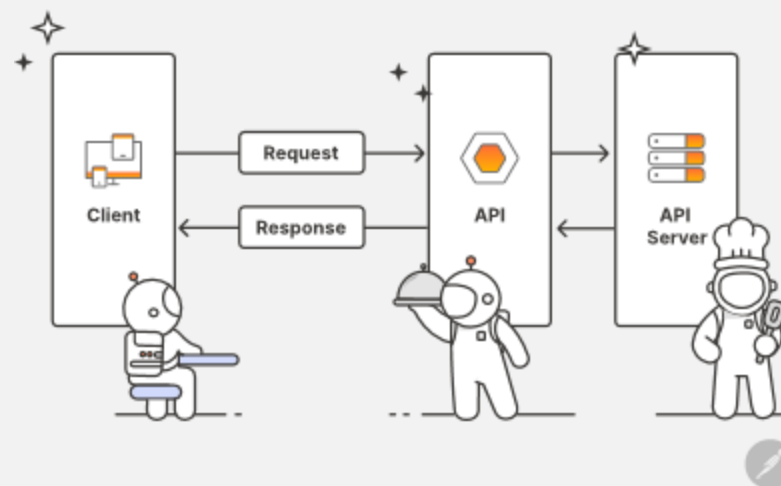
API



Definição

API significa **Application Programming Interface**, ou **Interface de Programação de Aplicações**. Em termos simples, uma API é um conjunto de regras e protocolos que **permite que diferentes softwares se comuniquem entre si**.

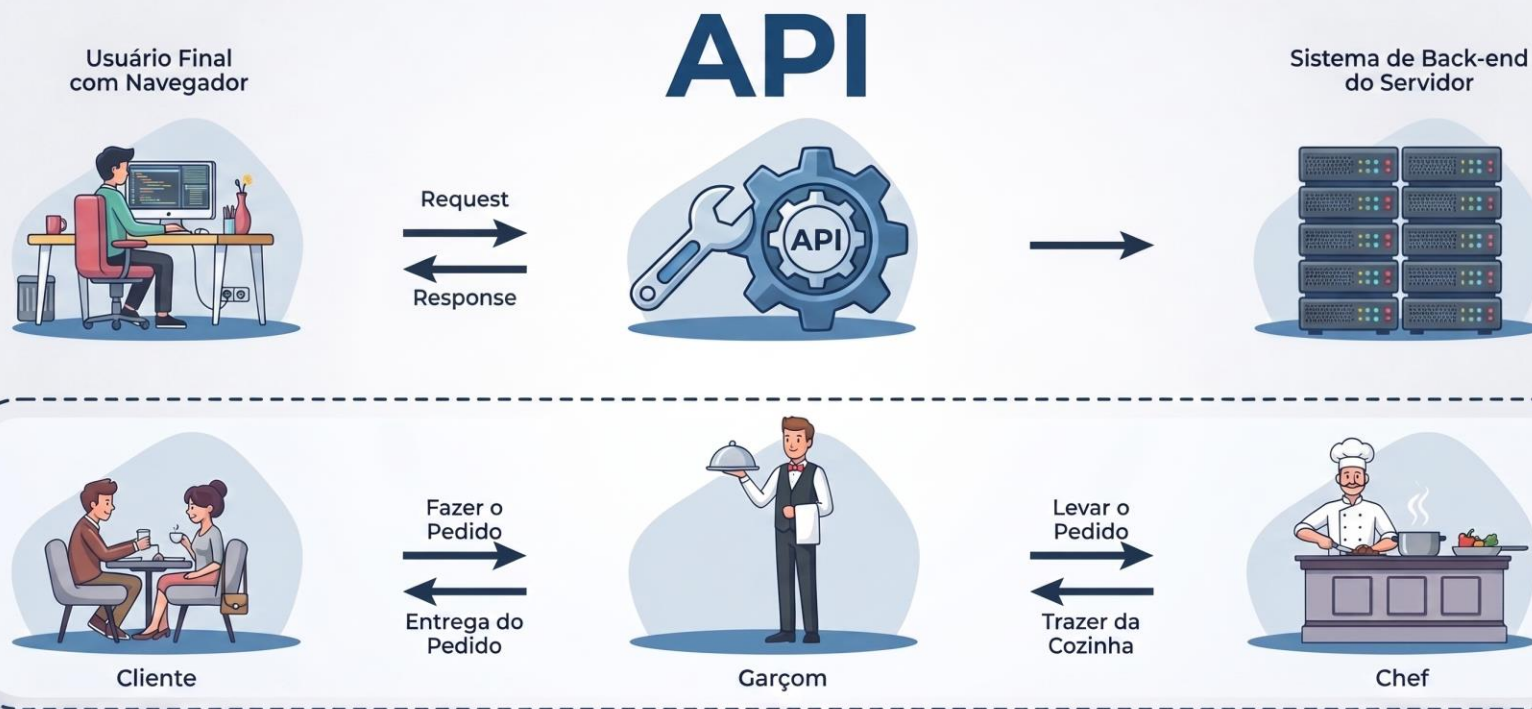
Ela funciona como uma **ponte**, facilitando a troca de dados e permitindo que aplicativos usem funcionalidades de terceiros sem precisar acessar o **código-fonte original**.



PROGRAMAÇÃO EM PYTHON



API



Pense em uma **API** como um **garçom** em um restaurante. **Você** (o cliente) não vai diretamente a cozinha preparar sua refeição. Em vez disso, você olha o cardápio (a documentação da **API**), faz seu pedido ao **garçom** (a requisição), e ele leva seu pedido a cozinha (o servidor) e traz sua comida (a resposta). O **garçom** é a **API** - o intermediário que facilita a comunicação.

PROGRAMAÇÃO EM PYTHON



API



Por que APIs são importantes ?

As **APIs** são fundamentais no mundo moderno da tecnologia por diversos motivos:

- **Integração:** Permitem que **sistemas diferentes trabalhem juntos**, mesmo que tenham sido criados em linguagens diferentes;
- **Reutilização:** Evitam que desenvolvedores recriem funcionalidades que já existem;
- **Eficiência:** Aceleram o desenvolvimento de aplicações;
- **Escalabilidade:** Permitem que serviços cresçam de forma modular.

Exemplos de APIs no Dia a Dia:

Você utiliza **APIs** constantemente, mesmo sem perceber:

- Google Maps: Aplicativos de entrega usam a **API** do Google para mostrar mapas;
- Previsão do Tempo: Sites de notícias consomem **APIs** meteorológicas;
- Pagamentos Online: Lojas virtuais usam **APIs** de gateways de pagamento;
- Login com Redes Sociais: O botão 'Entrar com Google' usa uma **API** de autenticação.

PROGRAMAÇÃO EM PYTHON



Como a Internet Funciona - HTTP



O Protocolo HTTP

HTTP (**H**ypertext **T**ransfer **P**rotocol) é o protocolo principal utilizado para comunicação na **World Wide Web**. Ele define como **mensagens são formatadas e transmitidas entre clientes** (navegadores, aplicativos) e servidores.

O funcionamento básico segue um modelo de requisição-resposta: o cliente envia uma requisição (**request**) e o servidor responde (**response**). Este ciclo é a base de toda comunicação web.

Métodos HTTP

Os métodos HTTP indicam a ação que desejamos realizar. Os principais são:

Método	Descrição
GET	Solicita dados do servidor. Apenas leitura, não altera dados.
POST	Envia dados para criar um novo recurso no servidor.
PUT	Atualiza completamente um recurso existente
PATCH	Atualiza parcialmente um recurso existente.
DELETE	Remove um recurso do servidor.

PROGRAMAÇÃO EM PYTHON



Como a Internet Funciona - HTTP



Códigos de Status HTTP

Toda resposta **HTTP** contém um **código de status** que indica o resultado da **requisição**. Eles são organizados em famílias numéricas:

Família	Significado
1xx	Informativo - a requisição foi recebida e esta sendo processada.
2xx	Sucesso - a requisição foi recebida, compreendida e aceita.
3xx	Redirecionamento - ações adicionais são necessárias.
4xx	Erro do Cliente - a requisição contém erro de sintaxe ou não pode ser processada.
5xx	Erro do Servidor - o servidor falhou ao processar uma requisição válida.

PROGRAMAÇÃO EM PYTHON



Como a Internet Funciona - HTTP



Anatomia de uma Requisição HTTP

Uma requisição **HTTP** completa possui as seguintes partes:

- **Linha de Requisição**: Contem o método, a **URL** e a versão do **HTTP**;
- **Cabeçalhos** (Headers): Informações adicionais sobre a requisição;
- **Corpo** (Body): Dados enviados ao servidor (opcional, usado em POST/PUT).

```
GET /api/usuarios HTTP/1.1
Host: exemplo.com
Accept: application/json
```

Anatomia de uma Resposta HTTP

Uma resposta **HTTP** também possui uma estrutura definida:

- **Linha de Status**: Versão do **HTTP**, código de status e mensagem;
- **Cabeçalhos** (Headers): Metadados sobre a resposta;
- **Corpo** (Body): Os dados propriamente ditos.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"nome": "Maria", "idade": 17}
```

PROGRAMAÇÃO EM PYTHON



JSON:

O Formato dos Dados



O que é JSON ?

JSON (**J**ava**S**cript **O**bject **N**otation) é um formato leve de **troca de dados**. É fácil de ler e escrever para humanos, e fácil de interpretar e gerar para máquinas. Apesar do nome, não é exclusivo do **JavaScript** - e amplamente suportado em praticamente todas as linguagens de programação, incluindo **Python**.

Estrutura do JSON:

O **JSON** é baseado em duas estruturas universais:

- **Objetos**: Coleções de pares chave-valor, delimitados por chaves {};
- **Arrays**: Listas ordenadas de valores, delimitados por colchetes [].

```
{
  "nome": "Joao Silva",
  "idade": 16,
  "cidade": "Sao Paulo",
  "notas": [8.5, 9.0, 7.5],
  "ativo": true
}
```

PROGRAMAÇÃO EM PYTHON



JSON:

O Formato dos Dados



Tipos de Dados em JSON

Tipo JSON	Exemplo	Descrição
string	"Hello"	Texto entre aspas duplas
number	42, 3.14	Números inteiros ou decimais
boolean	true, false	Valores lógicos
null	null	Valor nulo
array	[1, 2, 3]	Lista ordenada de valores
object	{"a": 1}	Coleção de pares chave-valor

Diferenças entre JSON e Python

JSON	Python
string	str
number	int ou float
boolean	bool
null	None
array	list
object	dict

PROGRAMAÇÃO EM PYTHON



JSON: O Formato dos Dados



JSON em Python

Python possui o modulo `json` para trabalhar com esse formato. A conversão entre **JSON** e objetos **Python** é direta:

- **JSON** para **Python** (parsing): `json.loads()` converte uma string **JSON** em dicionário/lista;
- **Python** para **JSON**: `json.dumps()` converte um dicionário/lista em string **JSON**.

```
import json

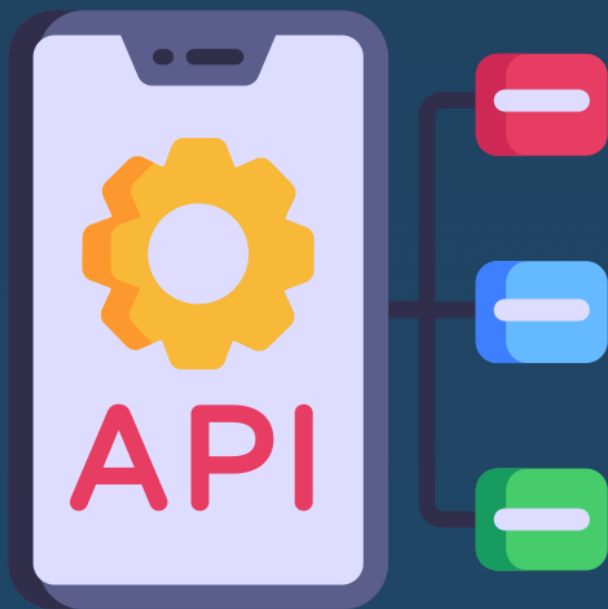
# JSON para Python (decodificar)
dados_json = '{"nome": "Ana", "idade": 15}'
dados_python = json.loads(dados_json)
print(dados_python['nome']) # Ana

# Python para JSON (codificar)
usuario = {'nome': 'Pedro', 'notas': [9, 8, 10]}
json_string = json.dumps(usuario, ensure_ascii=False)
print(json_string)
```

PROGRAMAÇÃO EM PYTHON



Tipos de APIs



APIs REST

REST (**R**epresentational **S**tate **T**ransfer) é o estilo arquitetural mais popular para **APIs** web. **APIs REST** usam os **métodos HTTP** de forma **padronizada** e **são stateless** (cada requisição é independente). Elas retornam dados geralmente em **JSON** ou **XML**.

Características principais:

- Usa **URLs** para identificar recursos (ex: /usuarios/123);
- Utiliza métodos **HTTP** semanticamente (**GET** para ler, **POST** para criar, etc.);
- Comunicação sem estado - **cada requisição contém todas as informações necessárias**.

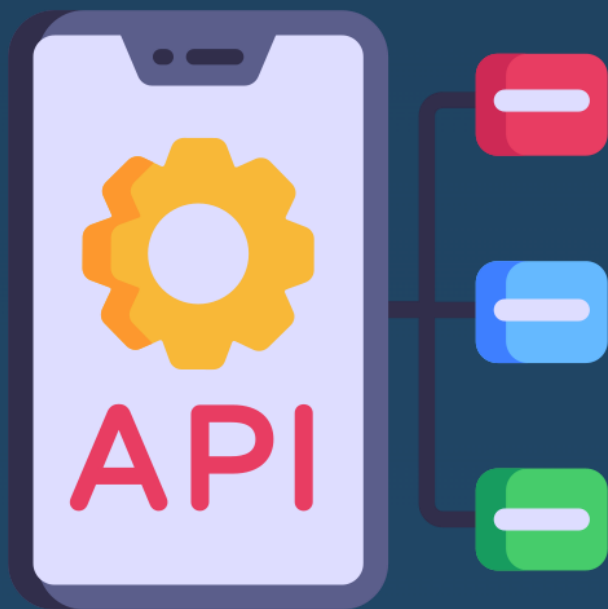
APIs SOAP

SOAP (**S**imple **O**bject **A**ccess **P**rotocol) é um protocolo mais antigo e rígido que usa **XML** para troca de mensagens. É mais complexo que REST, mas oferece maior **segurança** e **formalidade**. É comum em sistemas **corporativos** e legados.

PROGRAMAÇÃO EM PYTHON



Tipos de APIs



GraphQL

GraphQL é uma linguagem de consulta desenvolvida pelo Facebook. Ao contrário de REST, onde o servidor define o formato da resposta, em GraphQL o cliente especifica exatamente quais dados deseja receber, evitando transferência de dados desnecessários.

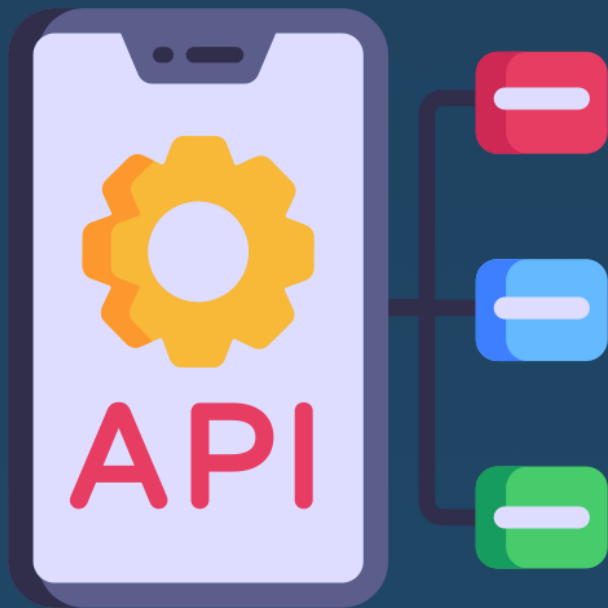
APIs Públicas vs Privadas

Tipo	Acesso	Exemplo
Pública	Livre, sem autenticação	PokeAPI, JSONPlaceholder
Privada	Requer chave/credencial	APIs de pagamento
Interna	Uso interno da empresa	Microserviços
Partner	Parceiros autorizados	APIs de fornecedores

PROGRAMAÇÃO EM PYTHON



Tipos de APIs



APIs REST

REST (**R**epresentational **S**tate **T**ransfer) é o estilo arquitetural mais popular para **APIs** web. **APIs REST** usam os **métodos HTTP** de forma **padronizada** e **são stateless** (cada requisição é independente). Elas retornam dados geralmente em **JSON** ou **XML**.

Características principais:

- Usa **URLs** para identificar recursos (ex: /usuarios/123);
- Utiliza métodos **HTTP** semanticamente (**GET** para ler, **POST** para criar, etc.);
- Comunicação sem estado - **cada requisição contém todas as informações necessárias**.

APIs SOAP

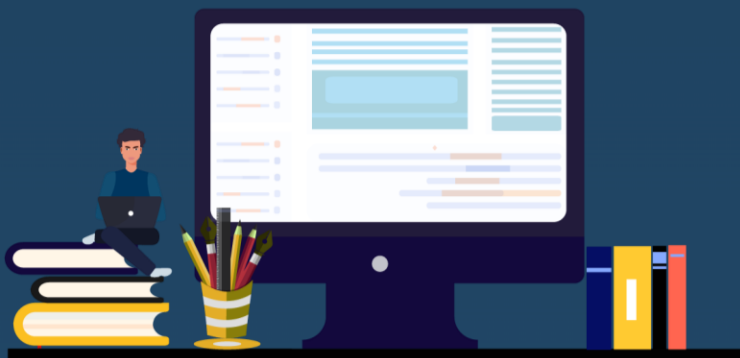
SOAP (**S**imple **O**bject **A**ccess **P**rotocol) é um protocolo mais antigo e rígido que usa **XML** para troca de mensagens. É mais complexo que REST, mas oferece maior **segurança** e **formalidade**. É comum em sistemas **corporativos** e legados.

PROGRAMAÇÃO EM PYTHON



Apresentação da situação de aprendizagem

1



Explorando APIs do Mundo Real - FABPROG

Imagine que você foi contratado pela empresa **FABPROG** para analisar uma API pública que será utilizada em um sistema digital. Antes do desenvolvimento da aplicação, é necessário compreender como a API funciona, quais informações ela fornece e como ocorre a comunicação entre cliente e servidor.

Para isso, você deverá realizar uma análise individual de uma API pública disponível na internet, compreendendo sua estrutura, endpoints e respostas em formato JSON.

Sugestões de APIs :

- API do IBGE; OpenWeatherMap; JSONPlaceholder; PokeAPI.

Atividades

- Acessar a documentação da API escolhida
- Identificar pelo menos 3 endpoints (URLs)
- Analisar um exemplo de resposta em JSON

PROGRAMAÇÃO EM PYTHON



Questões Mediadoras



- Explique com suas palavras o que é uma API e de três exemplos de uso no seu dia a dia.
- Qual a diferença entre os métodos GET e POST? Em que situação cada um deve ser usado?
- Um amigo fez uma requisição e recebeu o código 404. O que isso significa e como ele pode resolver?

Observação: Pode ser utilizado o site <https://sorteio.com/sorteio-de-nomes> para sortear alunos a responderem, se achar necessário.

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca urllib



A `urllib` é um pacote da biblioteca padrão do Python que reúne vários módulos para trabalhar com URLs. Os principais módulos são:

- `urllib.request`: Para abrir e ler URLs (fazer requisições);
- `urllib.parse`: Para analisar e construir URLs;
- `urllib.error`: Para tratamento de exceções específicas.

A vantagem de usar `urllib` é que ela não requer instalação adicional. A desvantagem é que seu uso pode ser mais verboso comparado a bibliotecas de terceiros como `requests`.

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca urllib



Exemplo Básico de GET

Vamos fazer nossa primeira requisição HTTP usando urllib. O exemplo abaixo consome a API JSONPlaceholder, que simula uma API REST completa e é perfeita para testes:

```
import urllib.request
import json

# URL da API de posts (publicacoes)
url = 'https://jsonplaceholder.typicode.com/posts/1'

# Faz a requisicao GET
resposta = urllib.request.urlopen(url)

# Le e decodifica os dados
dados = resposta.read().decode('utf-8')

# Converte de JSON para dicionario Python
post = json.loads(dados)

# Exibe os dados
print(f"Titulo: {post['title']}")
print(f"Conteudo: {post['body']}")
```

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca urllib



Entendendo o Código Passo a Passo

- `import urllib.request`: Importa o modulo necessário para fazer requisições;
- `urlopen(url)`: Abre a conexão com o servidor e envia a requisição GET;
- `read()`: Le os bytes da resposta;
- `decode('utf-8')`: Converte bytes para string usando codificação UTF-8;
- `json.loads()`: Transforma a string JSON em um dicionário Python.

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca requests



Por que usar requests ?

A biblioteca requests foi criada com o lema '**HTTP** for Humans' (**HTTP** para Humanos). Ela abstrai muitas complexidades do **urllib** e oferece uma **API** muito mais intuitiva. Comparação rápida:

Operação	urllib	requests
Import necessário	3 módulos	1 módulo
Decodificar JSON	json.loads() manual	.json() automático
Passar parâmetros	urlencode manual	params = dict
Enviar JSON (POST)	encode() manual	json = dict
Tratamento de erros	Múltiplas exceções	raise_for_status()

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca requests



Instalação

A biblioteca requests não vem instalada por padrao. Para instala-la, use o pip:



```
pip install requests
```

Primeira Requisição com requests

Veja como o mesmo código fica mais simples com requests:



```
import requests

# Requisicao GET simples
resposta =
requests.get('https://jsonplaceholder.typicode.com/posts/1')

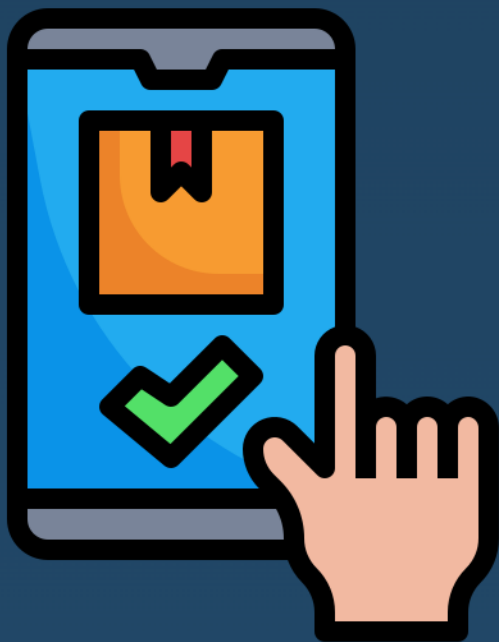
# A resposta ja e interpretada automaticamente
print(f"Status: {resposta.status_code}")
print(f"Tipo de conteudo: {resposta.headers['Content-Type']}")

# JSON ja vem como dicionario Python - sem necessidade de
json.loads()!
dados = resposta.json()
print(f"Titulo: {dados['title']}")
print(f"Conteudo: {dados['body']}")
```

PROGRAMAÇÃO EM PYTHON



Introdução a Biblioteca requests



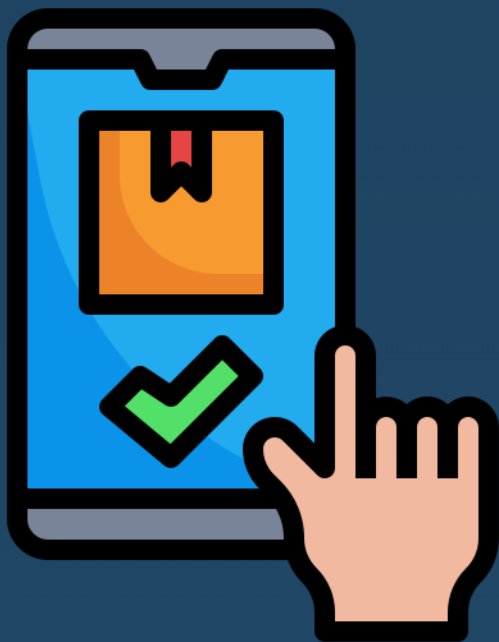
Vantagens do requests

- Não precisa importar vários módulos separados;
- JSON é decodificado automaticamente com `.json()`;
- Parâmetros de URL são passados como dicionário;
- Codificação de caracteres é tratada automaticamente;
- Sintaxe mais próxima da linguagem natural.

PROGRAMAÇÃO EM PYTHON



Métodos HTTP com requests



GET com Parâmetros

Com requests, passar parâmetros na URL e muito simples:

```
import requests

# Parametros como dicionario
parametros = {'userId': 1, '_limit': 5}

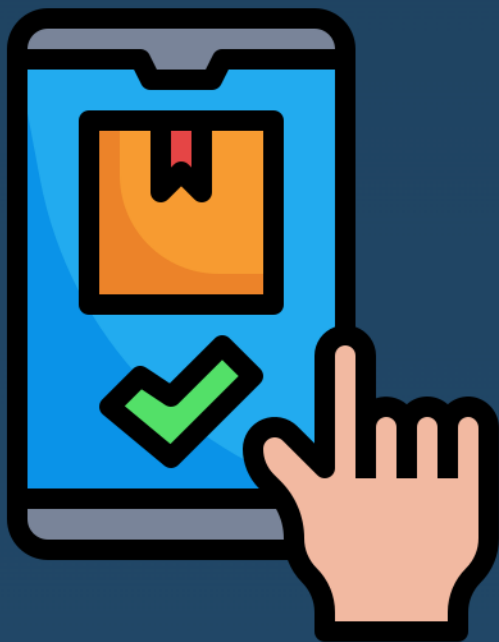
resposta = requests.get(
    'https://jsonplaceholder.typicode.com/posts',
    params=parametros
)

posts = resposta.json()
print(f"Retornados: {len(posts)} posts")
for post in posts:
    print(f"- {post['title'][:40]}...")
```

PROGRAMAÇÃO EM PYTHON



Métodos HTTP com requests



POST com JSON

Enviar dados JSON com requests é extremamente simples:

```
import requests

url = 'https://jsonplaceholder.typicode.com/posts'

# Dados a enviar - requests converte automaticamente para JSON
novo_post = {
    'title': 'Aprendendo requests',
    'body': 'E muito mais facil que urllib!',
    'userId': 1
}

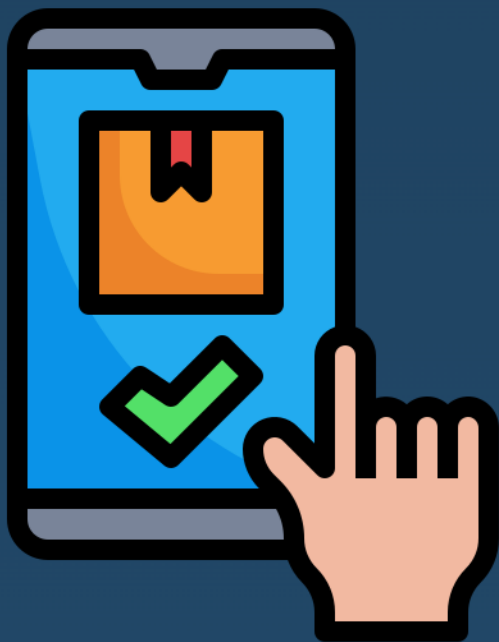
resposta = requests.post(url, json=novo_post)

print(f"Status: {resposta.status_code}")
print(f"Dados retornados: {resposta.json()}")
```

PROGRAMAÇÃO EM PYTHON



Métodos HTTP com requests



Outros Métodos

A biblioteca requests suporta todos os métodos HTTP de forma consistente:

```
import requests

base_url = 'https://jsonplaceholder.typicode.com/posts/1'

# PUT - Atualizar completamente
dados_atualizados = {
    'id': 1,
    'title': 'Titulo Atualizado',
    'body': 'Novo conteudo',
    'userId': 1
}
resp_put = requests.put(base_url, json=dados_atualizados)
print(f"PUT status: {resp_put.status_code}")

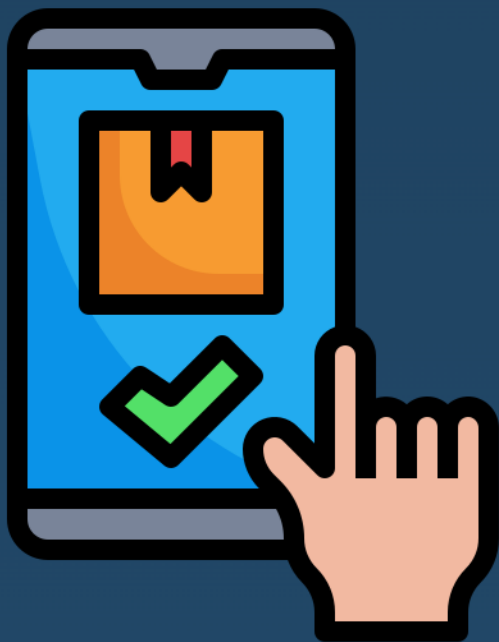
# PATCH - Atualizar parcialmente
resp_patch = requests.patch(base_url, json={'title': 'Apenas o titulo'})
print(f"PATCH status: {resp_patch.status_code}")

# DELETE - Remover
resp_delete = requests.delete(base_url)
print(f"DELETE status: {resp_delete.status_code}")
```

PROGRAMAÇÃO EM PYTHON



Métodos HTTP com requests



Resumo dos Métodos

Método	Função	Uso
GET	<code>requests.get()</code>	Obter dados
POST	<code>requests.post()</code>	Criar recurso
PUT	<code>requests.put()</code>	Atualizar completo
PATCH	<code>requests.patch()</code>	Atualizar parcial
DELETE	<code>requests.delete()</code>	Remover recurso



Autenticação em APIs



Tipos de Autenticação

Muitas **APIs** exigem autenticação para controlar o acesso. Os métodos mais comuns são:

- **API Key**: Uma chave única enviada em cada requisição;
- **Basic Auth**: Usuário e senha codificados em Base64;
- **Bearer Token** (OAuth 2.0): Token de acesso obtido previamente;
- **OAuth**: Protocolo completo de autorização.

PROGRAMAÇÃO EM PYTHON



Autenticação em APIs



Autenticação com API Key



```
import requests

url = 'https://api.exemplo.com/dados'
api_key = 'sua_chave_aqui'

# Metodo 1: Header
cabecalhos = {'Authorization': f'Bearer {api_key}'}
resposta = requests.get(url, headers=cabecalhos)

# Metodo 2: Parametro de URL
resposta = requests.get(url, params={'api_key': api_key})
```

PROGRAMAÇÃO EM PYTHON



Autenticação em APIs



Autenticação Básica (Basic Auth)

```
import requests
from requests.auth import HTTPBasicAuth

url = 'https://api.exemplo.com/protegido'

# Basic Auth com usuario e senha
resposta = requests.get(url, auth=HTTPBasicAuth('usuario',
'senha'))

# Atalho - passando tupla diretamente
resposta = requests.get(url, auth=('usuario', 'senha'))
```

PROGRAMAÇÃO EM PYTHON



Tratamento de Erros



Verificação de Erros com `raise_for_status`

O requests oferece um método elegante para tratar erros HTTP:

```
import requests

url = 'https://jsonplaceholder.typicode.com/posts/999999'

try:
    resposta = requests.get(url)
    resposta.raise_for_status() # Lança exceção se status >= 400
    dados = resposta.json()
    print(dados['title'])

except requests.exceptions.HTTPError as e:
    print(f"Erro HTTP: {e}")

except requests.exceptions.ConnectionError:
    print("Erro: Não foi possível conectar ao servidor")

except requests.exceptions.Timeout:
    print("Erro: A requisição excedeu o tempo limite")

except requests.exceptions.RequestException as e:
    print(f"Erro na requisição: {e}")
```

PROGRAMAÇÃO EM PYTHON



Tratamento de Erros



Definindo Timeout

E importante sempre definir um timeout para evitar que o programa fique esperando indefinidamente:

```
import requests

try:
    # Timeout de 5 segundos
    resposta = requests.get(
        'https://jsonplaceholder.typicode.com/posts',
        timeout=5
    )
    print(f"OK: {len(resposta.json())} posts")

except requests.exceptions.Timeout:
    print("O servidor demorou muito para responder")
```

PROGRAMAÇÃO EM PYTHON



Tratamento de Erros



Tabela de Exceções do requests

Exceção	Quando Ocorre
ConnectionError	Falha de conexão de rede
HTTPError	Resposta HTTP com erro (4xx, 5xx)
Timeout	Tempo limite excedido
TooManyRedirects	Muitos redirecionamentos
RequestException	Classe base - captura qualquer erro

PROGRAMAÇÃO EM PYTHON



Projeto Prático Integrador

Vamos
realizar
juntos!



Descrição do Projeto

Para consolidar todo o aprendizado, vamos desenvolver um Consultor de Clima completo. Este projeto integra todos os conceitos vistos nos últimos dias de estudo.

Especificações do Projeto

Desenvolva um programa Python que:

- Consulte a API Open-Meteo (gratuita, sem chave) para obter dados meteorológicos;
- Receba do usuário o nome de uma cidade;
- Converta o nome da cidade em coordenadas usando a API de Geocoding do Open-Meteo;
- Obtenha a previsão do tempo atual para essas coordenadas;
- Exiba de forma amigável: temperatura, sensação térmica, umidade e velocidade do vento.

PROGRAMAÇÃO EM PYTHON



Projeto Prático Integrador

Vamos
realizar
juntos!



```
import requests
```

```
def obter_coordenadas(cidade):  
    """Obtem latitude e longitude de uma cidade."""  
    url = 'https://geocoding-api.open-meteo.com/v1/search'  
    params = {'name': cidade, 'count': 1, 'language': 'pt', 'format': 'json'}  
  
    resposta = requests.get(url, params=params, timeout=5)  
    resposta.raise_for_status()  
  
    dados = resposta.json()  
    if 'results' not in dados or not dados['results']:  
        return None  
  
    resultado = dados['results'][0]  
    return {  
        'nome': resultado['name'],  
        'pais': resultado.get('country', 'Desconhecido'),  
        'latitude': resultado['latitude'],  
        'longitude': resultado['longitude']  
    }
```

PROGRAMAÇÃO EM PYTHON



Projeto Prático Integrador

Vamos
realizar
juntos!



```
def obter_clima(latitude, longitude):  
    """Obtem dados climaticos atuais."""  
    url = 'https://api.open-meteo.com/v1/forecast'  
    params = {  
        'latitude': latitude,  
        'longitude': longitude,  
        'current':  
        'temperature_2m,relative_humidity_2m,apparent_temperature,wind_s  
peed_10m',  
        'timezone': 'auto'  
    }  
  
    resposta = requests.get(url, params=params, timeout=5)  
    resposta.raise_for_status()  
    return resposta.json()['current']
```


PROGRAMAÇÃO EM PYTHON



Projeto Prático Integrador

Vamos
realizar
juntos!



```
def main():
    print('=== Consultor de Clima ===')
    cidade = input('Digite o nome de uma cidade: ')

    coords = obter_coordenadas(cidade)
    if not coords:
        print(f"Cidade '{cidade}' nao encontrada.")
        return

    print(f"Localizando: {coords['nome']}, {coords['pais']}")

    clima = obter_clima(coords['latitude'], coords['longitude'])

    print('\n--- Dados Atuais ---')
    print(f"Temperatura: {clima['temperature_2m']} C")
    print(f"Sensacao termica: {clima['apparent_temperature']} C")
    print(f"Umididade: {clima['relative_humidity_2m']}%")
    print(f"Vento: {clima['wind_speed_10m']} km/h")

if __name__ == '__main__':
    main()
```

REFERÊNCIAS

Documentação oficial do Python - urllib:
<https://docs.python.org/3/library/urllib.html>

MENEZES, B. N. Introdução a Programação com Python. 3. ed. São Paulo: Novatec, 2019.

Documentação da biblioteca requests:
<https://requests.readthedocs.io/>

JSONPlaceholder - API para testes:
<https://jsonplaceholder.typicode.com/>

Open-Meteo API - Dados meteorológicos: <https://open-meteo.com/>

REST API Tutorial: <https://restfulapi.net/>

Importante:

Os conteúdos disponibilizados são específicos para este curso/turma, a divulgação ou reprodução do material para outras pessoas/organização não é autorizada.



wilck.gomes@sp.senai.br



Wilck Gomes de Oliveira



Instrutor de Formação Profissional III

